Session #12:

Image Segmentation: Metrics and Architecture

Mithun Kumar S R
mithunkumar.sr@wilp.bits-pilani.ac.in

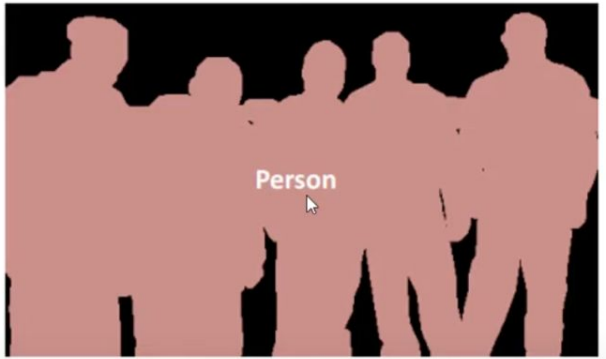
Acknowledgement: Slide Materials adopted from - EECS 442 – David Fouhey and prepared by the lead instructor Prof S P Vimal

Agenda

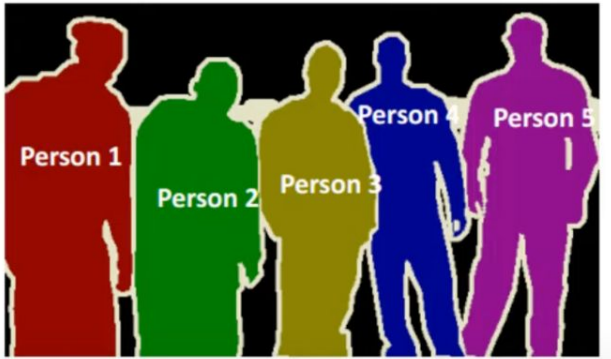
Metrics for Object Segmentation
Popular DNN Architectures for Segmentation

Segmentation vs Detection

Object Detection

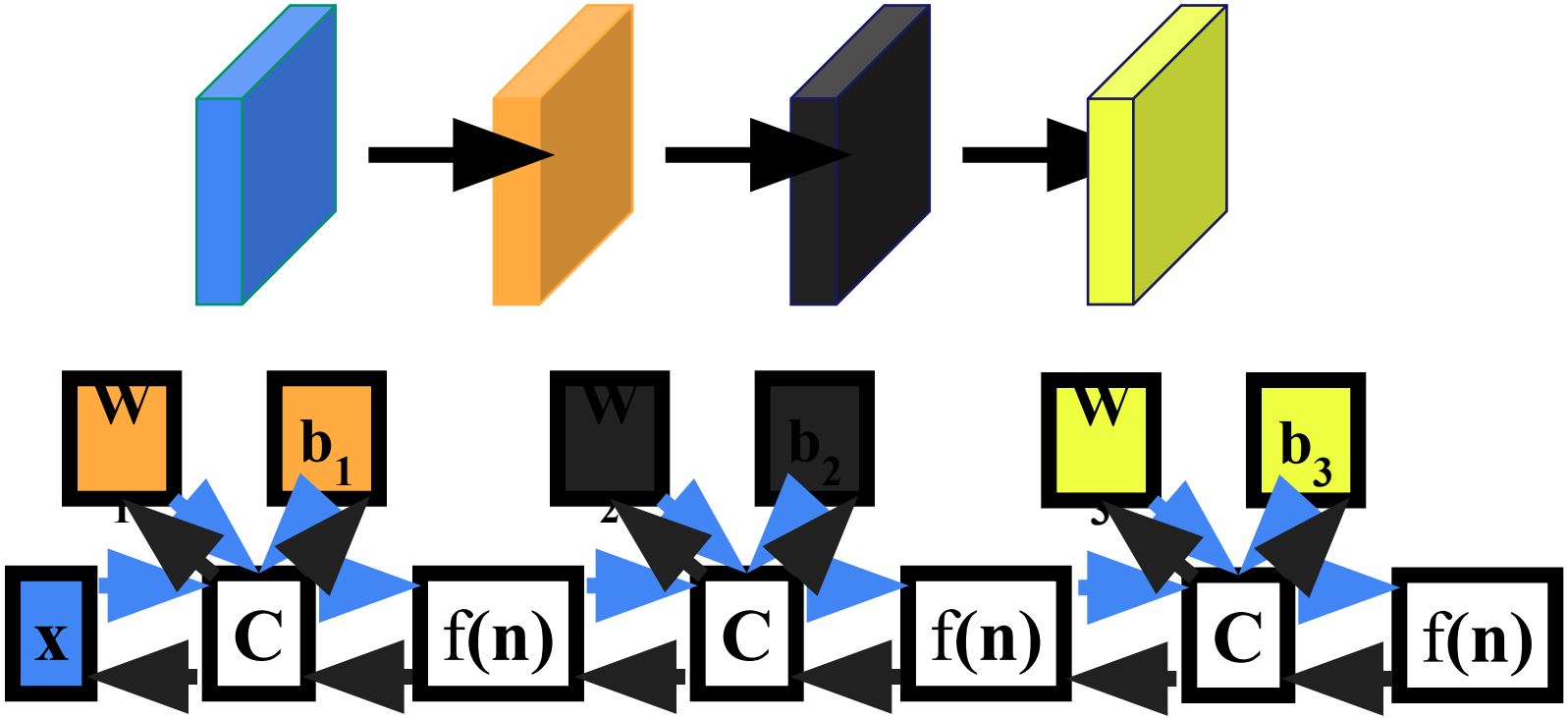


Semantic Segmentation

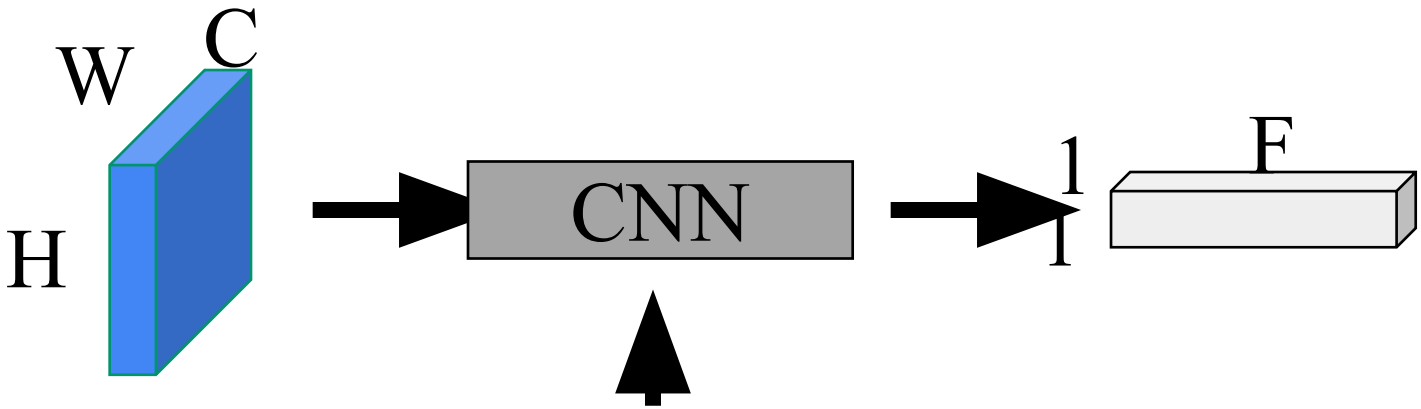


Instance Segmentation

Convolutional Neural Network (CNN)

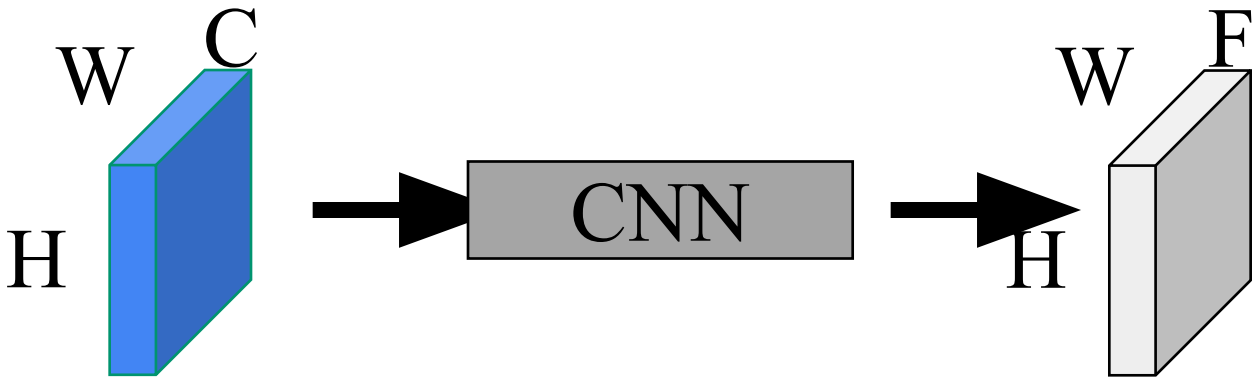


CNN



Function of the image that is parameterized by the convolutional filter weights and biases. We design the form of the function and fit the parameters to data.

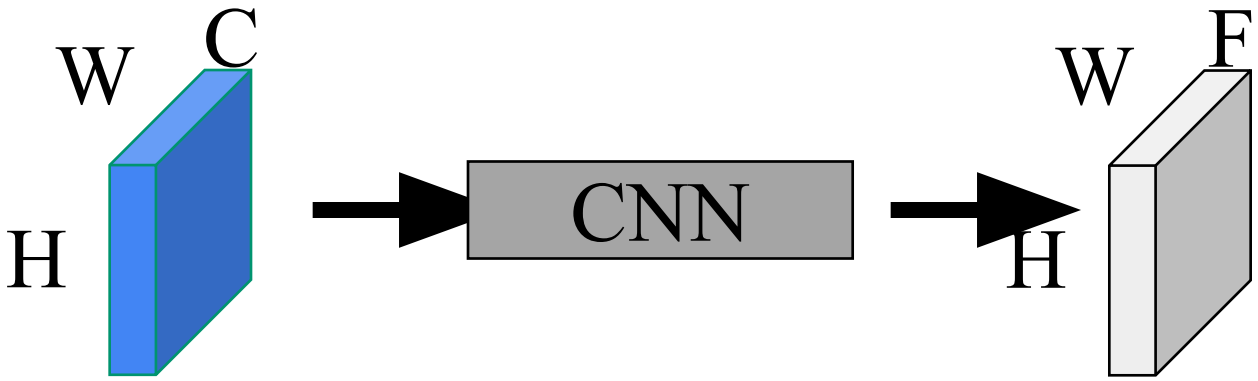
Now



<u>Convert HxW image into a F-dimensional vector</u>

Which pixels in this image are a cat?
How far is each pixel away from the camera?
Which pixels of this image are fake?

Semantic Segmentation



<u>Today's Running Example</u>

- Predict F-dimensional vector representing probability of each of F classes at every pixel
  - Loss computed/backprop'd at *every* pixel.

Semantic Segmentation

Each pixel has label

Usually visualized by colors.

Note: don’t distinguish between object *instances*

Input Label

Input Label

Image Credit: Everingham et al. Pascal VOC 2012.

Semantic Segmentation

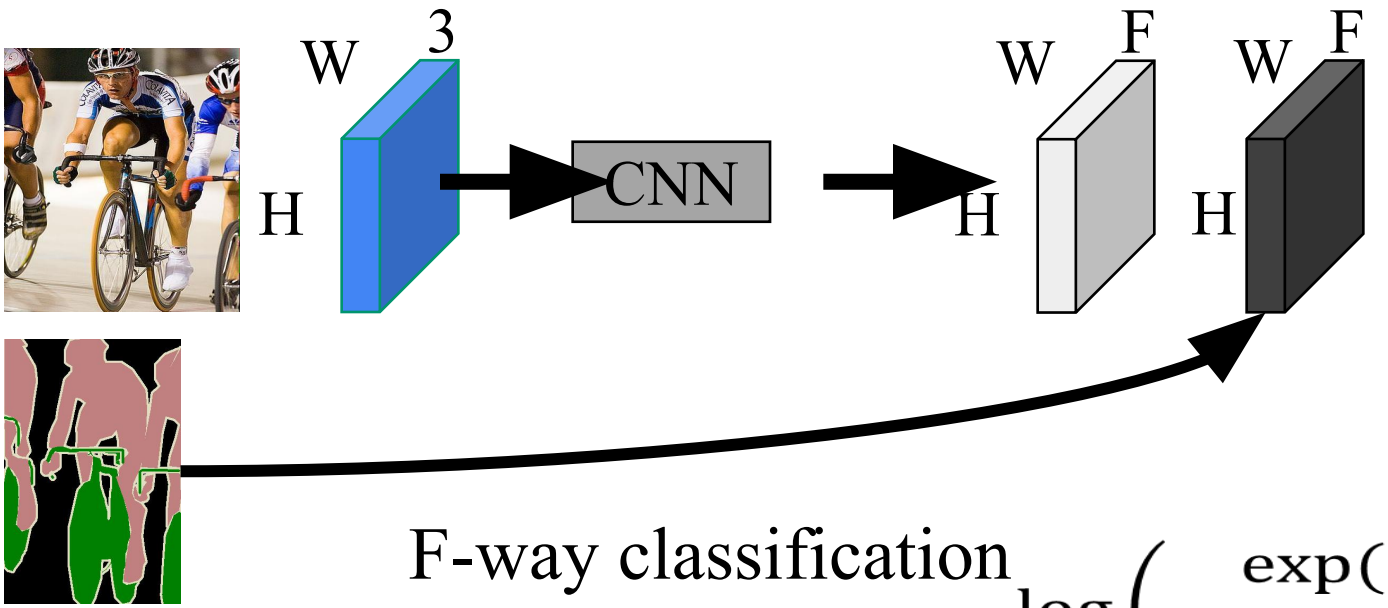
"Semantic": a usually meaningless word and an indication that someone is trying to trick you.

Meant to indicate here that we're **naming** things.

Input Label

Input Label

Semantic Segmentation



F-way classification loss function at every pixel:

$$-\log\left(\frac{\exp((Wx)_{y_i}}{\sum_k \exp((Wx)_k))}\right)$$

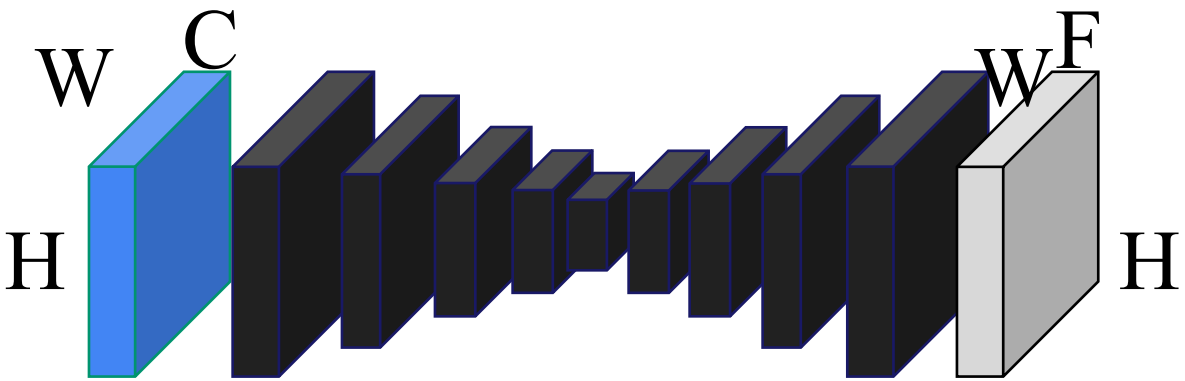
Image Credit: Everingham et al. Pascal VOC 2012.

Encoder-Decoder

Key idea: First **downsample** towards middle of network. Then **upsample** from middle.

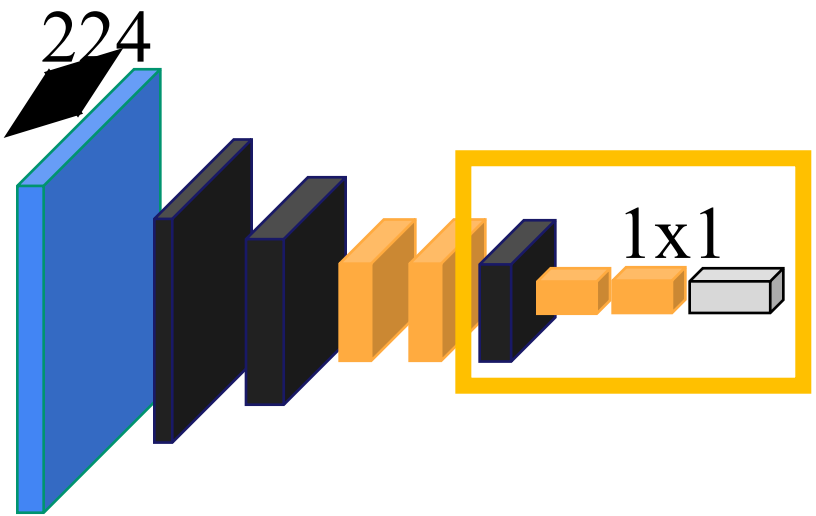
How do we downsample?

Convolutions, pooling



Where Do We Get Parameters?

Convnet that maps images to vectors

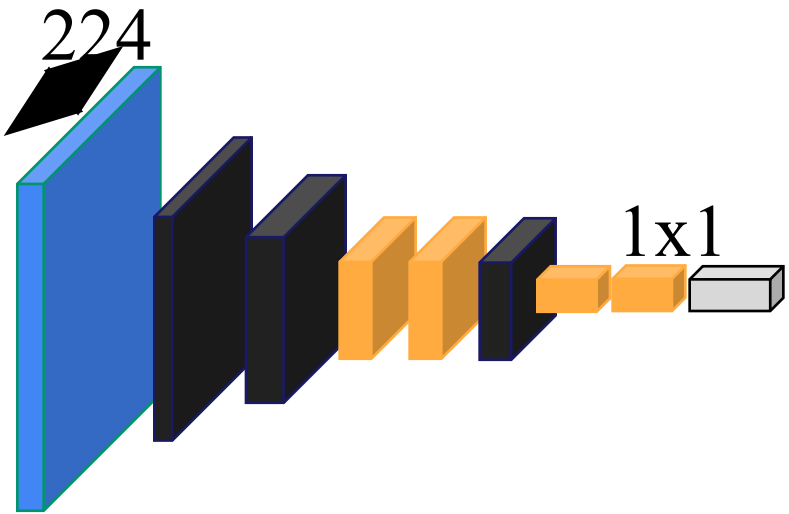


Recall that we can rewrite any vector-vector operations via 1x1 convolutions

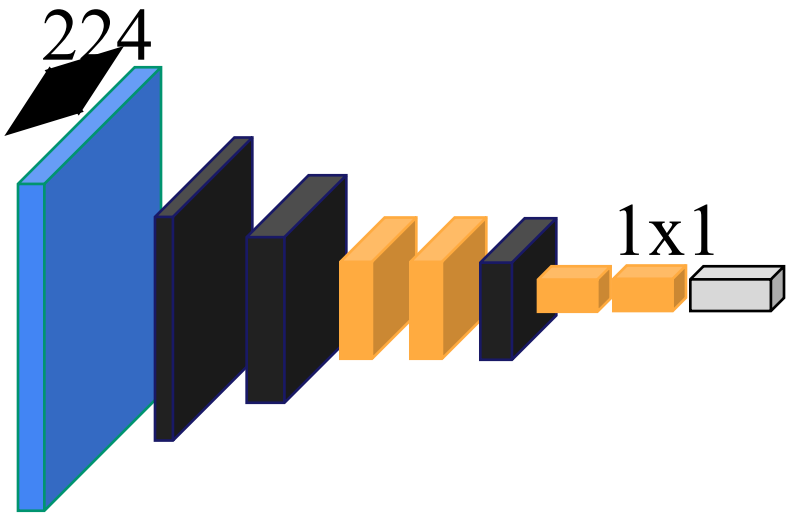
Long et al. *Fully Convolutional Networks For Semantic Segmentation*. CVPR 2014

Where Do We Get Parameters?

Convnet that maps images to vectors



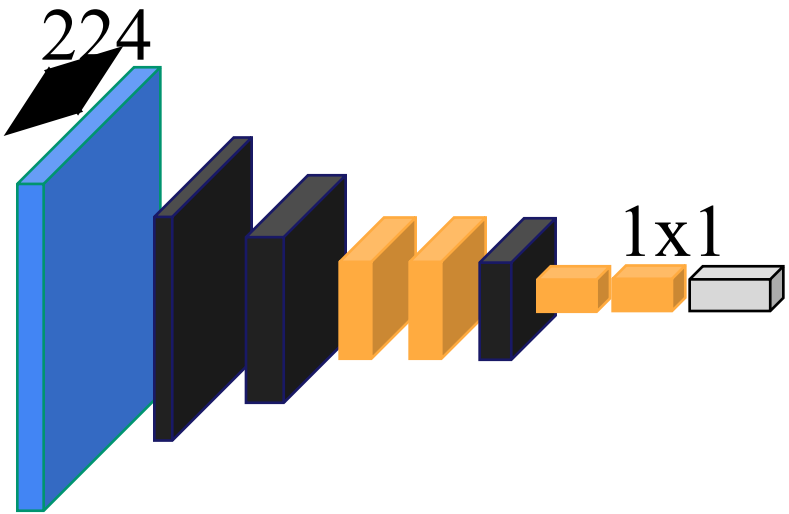
Convnet that maps images to images



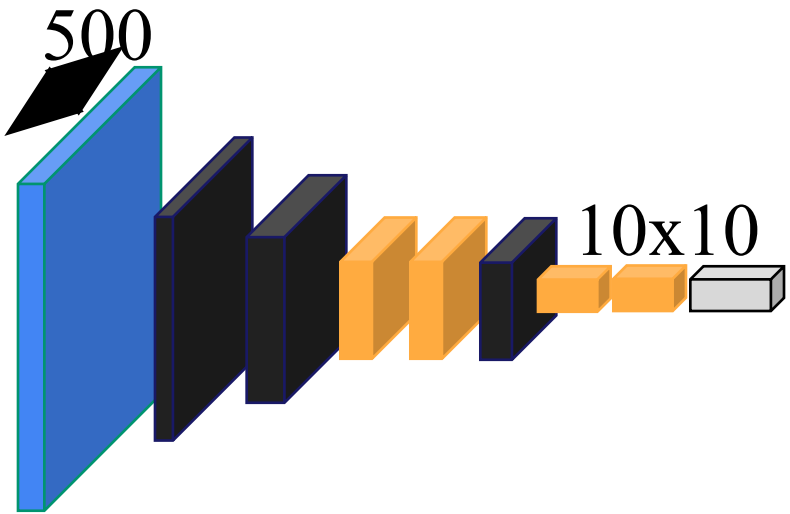
What if we make the input bigger?

Where Do We Get Parameters?

Convnet that maps images to vectors

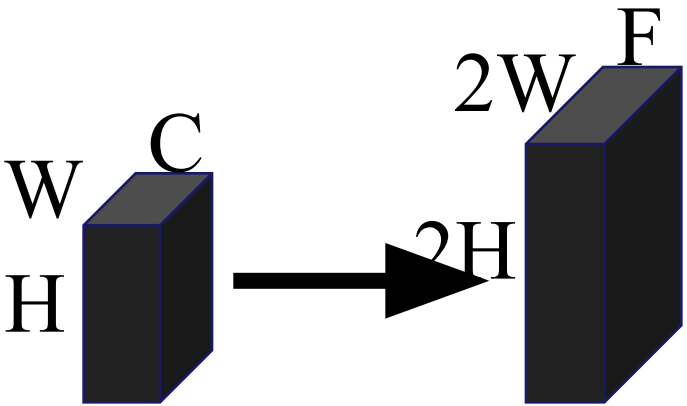


Convnet that maps images to images



Since it's convolution, can reuse an image network

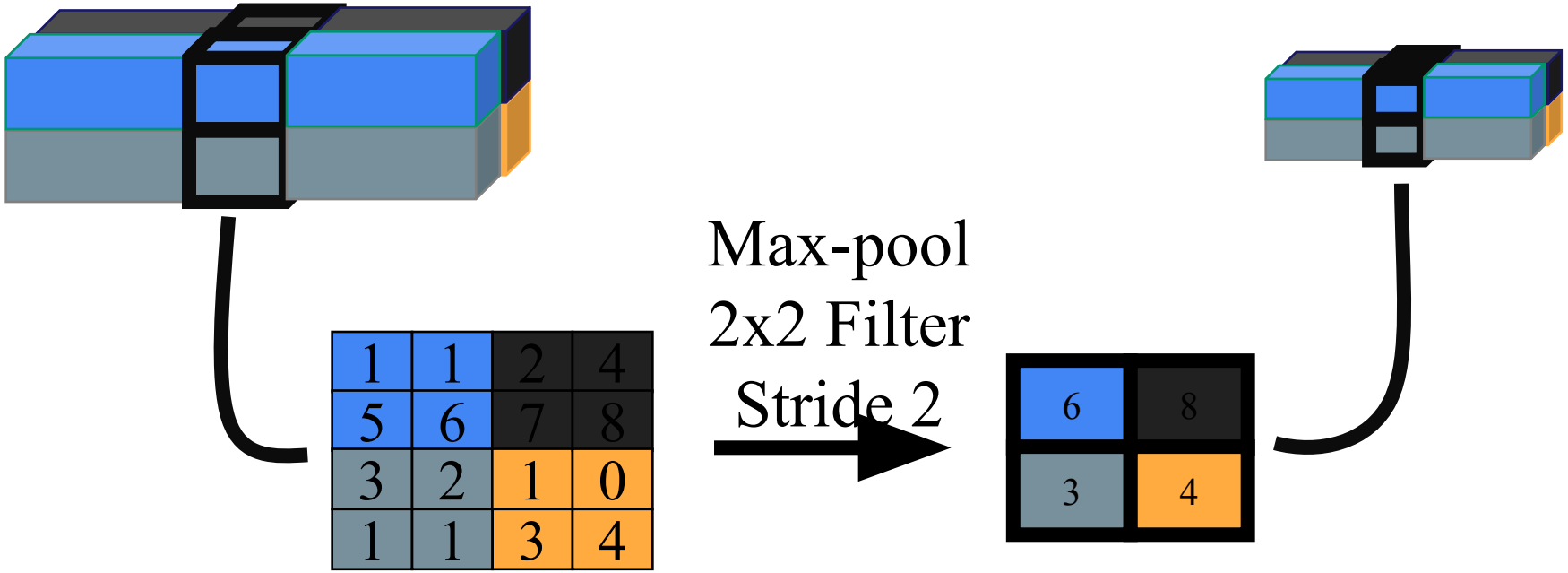
How Do We Upsample?



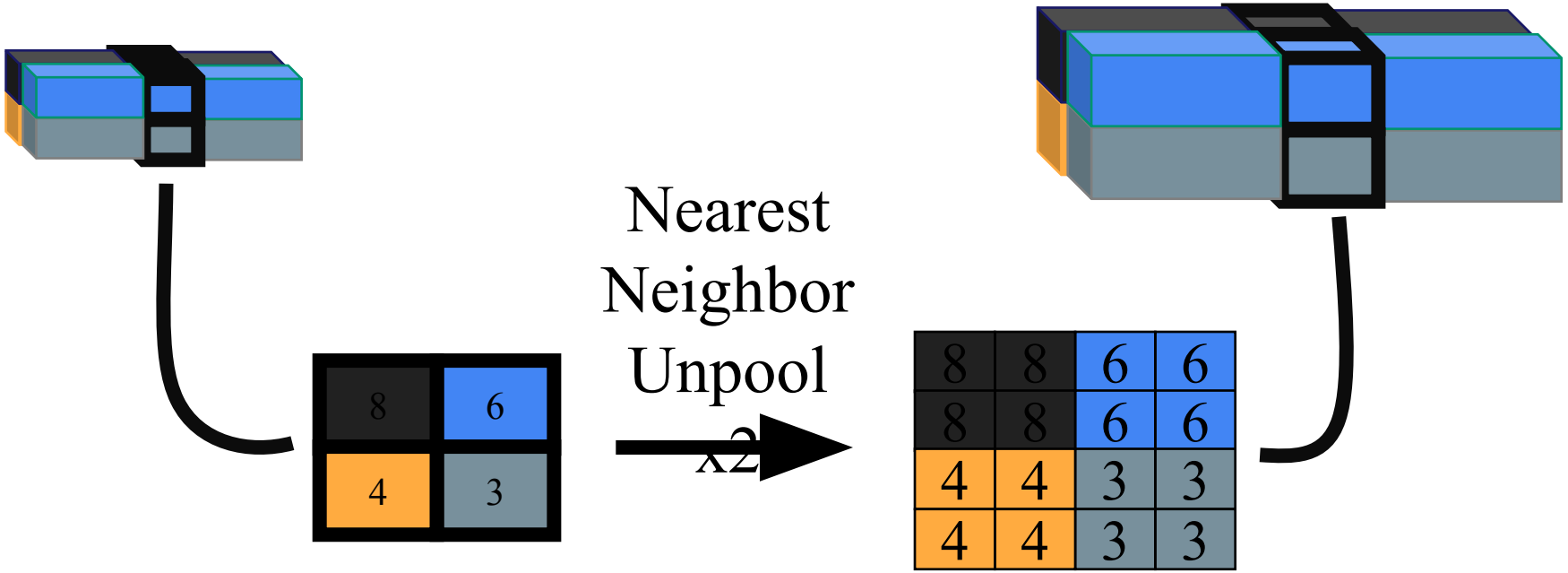
Do the opposite of how we downsample:

1. Pooling → "Unpooling"
2. Convolution → "Transpose Convolution"

Recall: Pooling

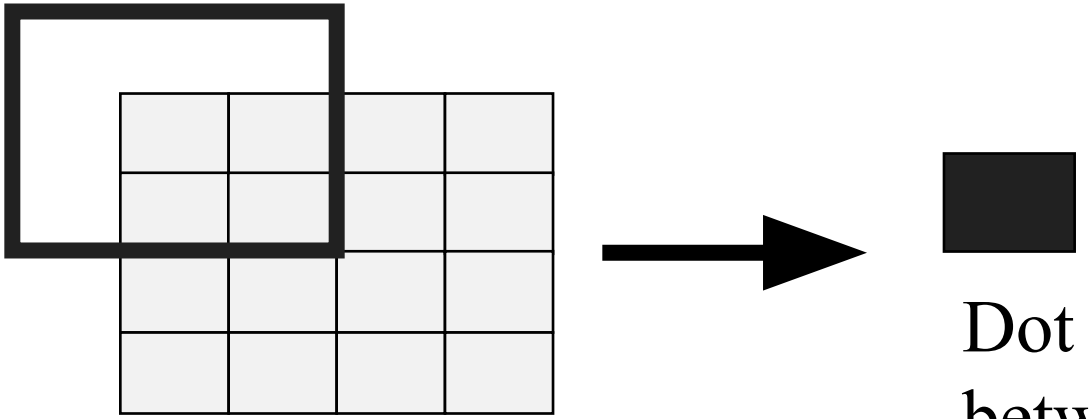


Now: Unpooling



Recall: Convolution

3x3 Convolution, Stride 2, Pad 1

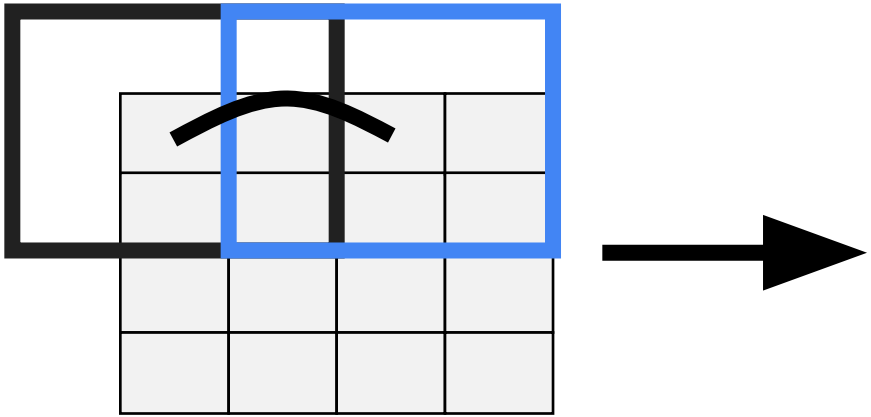


Dot product between filter f and input

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Recall: Convolution

3x3 Convolution, Stride 2, Pad 1

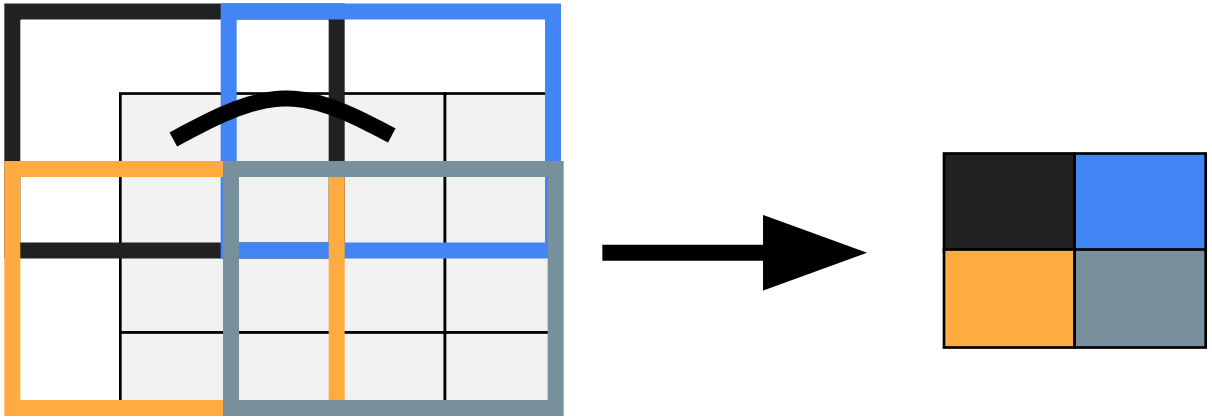


Dot product between filter f and input

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Recall: Convolution

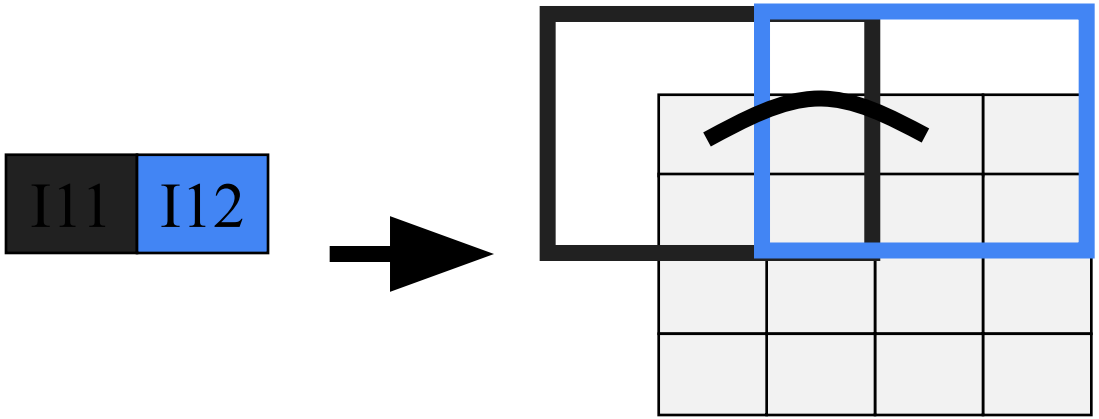
3x3 Convolution, Stride 2, Pad 1



Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Transpose Convolution

3x3 Transpose Convolution, Stride 2, Pad 1



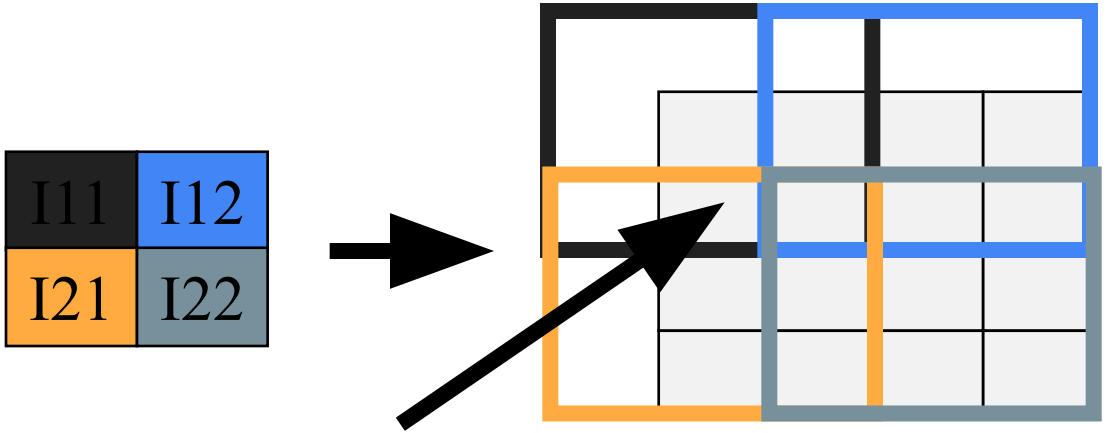
Output is filter F weighted by input I

$I_{11}F$ $I_{12}F$

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Transpose Convolution

3x3 Transpose Convolution, Stride 2, Pad 1



Output is filter F weighted by input I

$I_{11}F$ $I_{12}F$

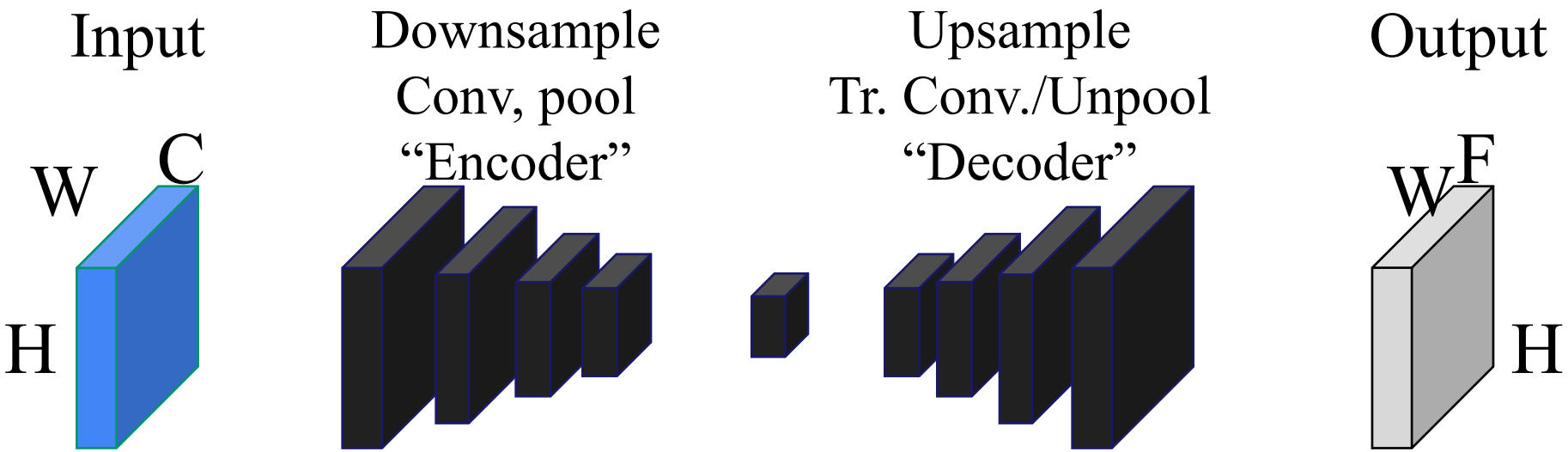
$I_{21}F$ $I_{22}F$

Sum outputs at overlap (e.g., from $I_{11}F$ and $I_{21}F$)

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Putting it Together

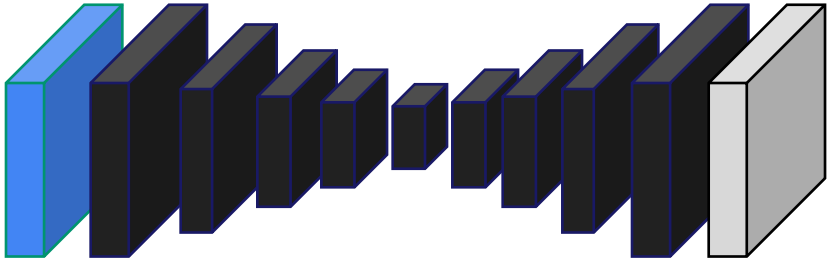
Convolutions + pooling downsample/compress/encode
Transpose convs./unpoolings upsample/uncompress/decode



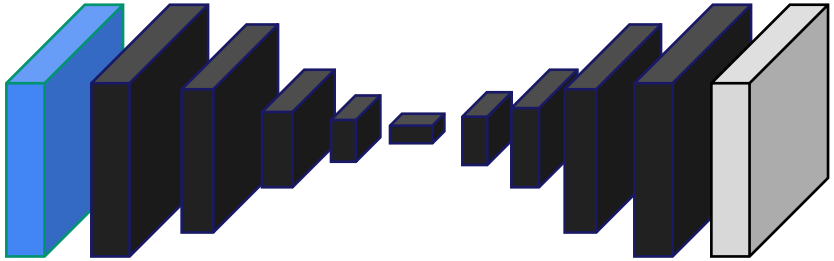
Putting It Together – Block Sizes

- Networks come in lots of forms
- **Don't take any block sizes literally.**
- Often (not always) keep some spatial resolution

Encode to spatially smaller tensor, then decode.

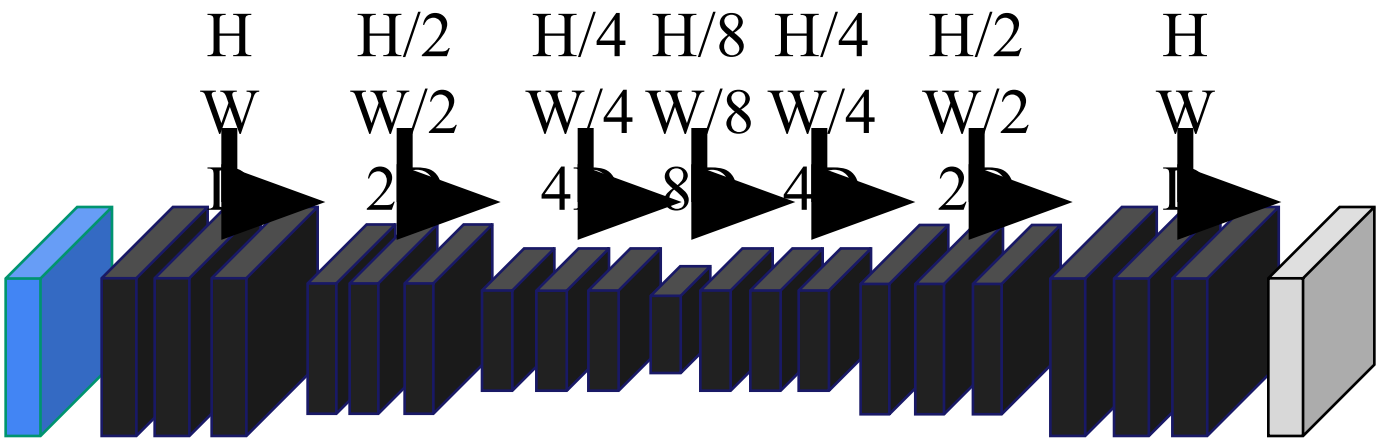


Encode to 1D vector then decode



Putting It Together – Block Sizes

- Often multiple layers at each spatial resolution.
- Often halve spatial resolution and double feature depth every few layers

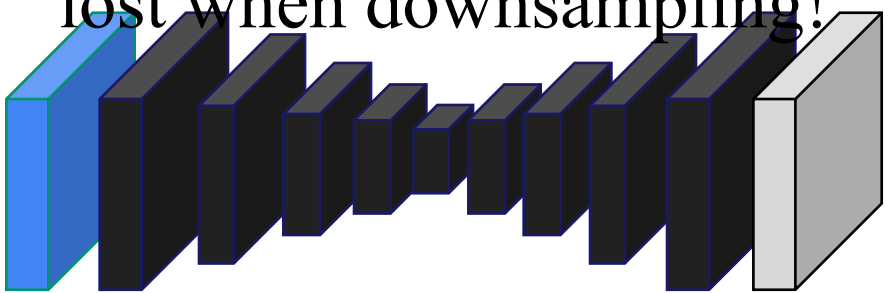


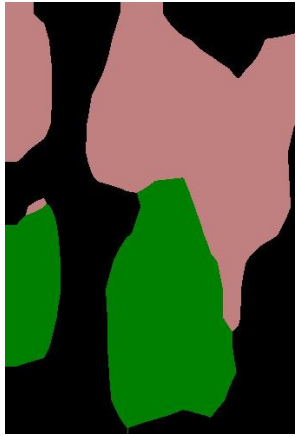
Missing Details

While the output *is* HxW, just upsampling often produces results without details/not aligned with the image.

Why?

Information about details lost when downsampling!

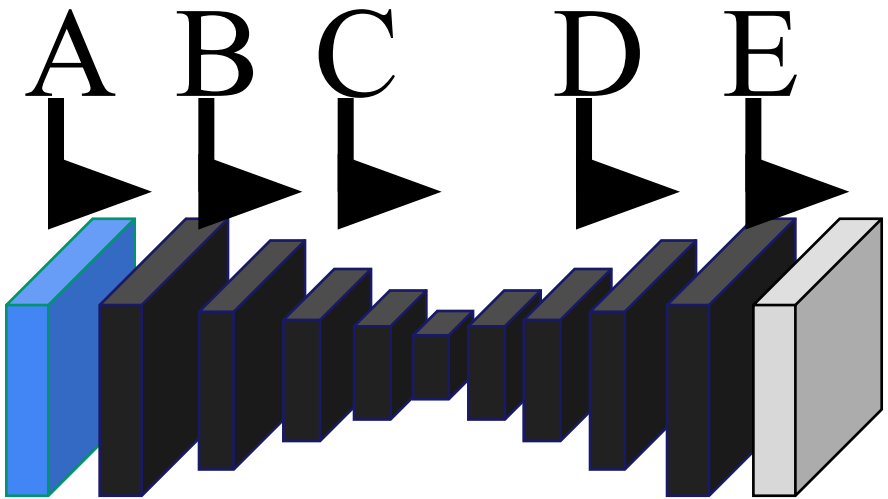


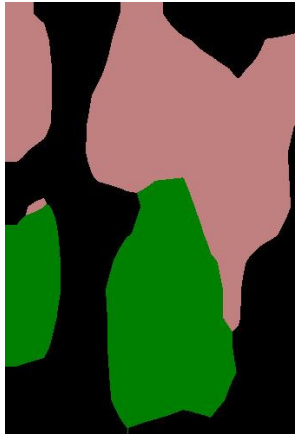


Result from Long et al. *Fully Convolutional Networks For Semantic Segmentation*. CVPR 2014

Missing Details

Where is the useful information about the high-frequency details of the image?

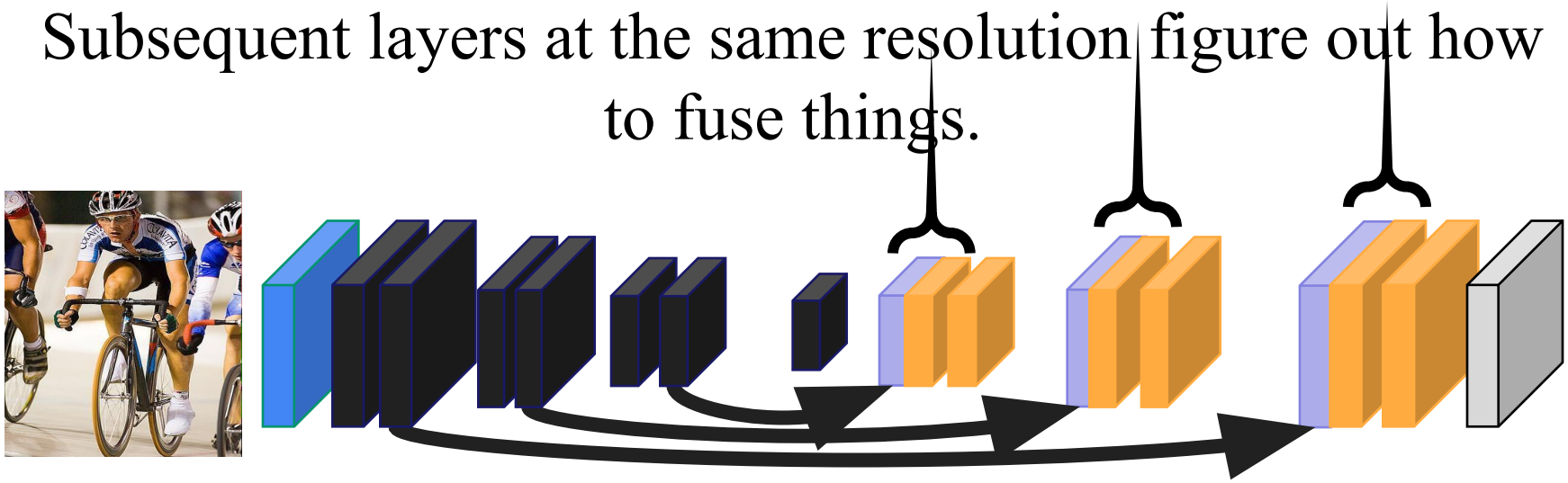




Result from Long et al. *Fully Convolutional Networks For Semantic Segmentation*. CVPR 2014

Missing Details

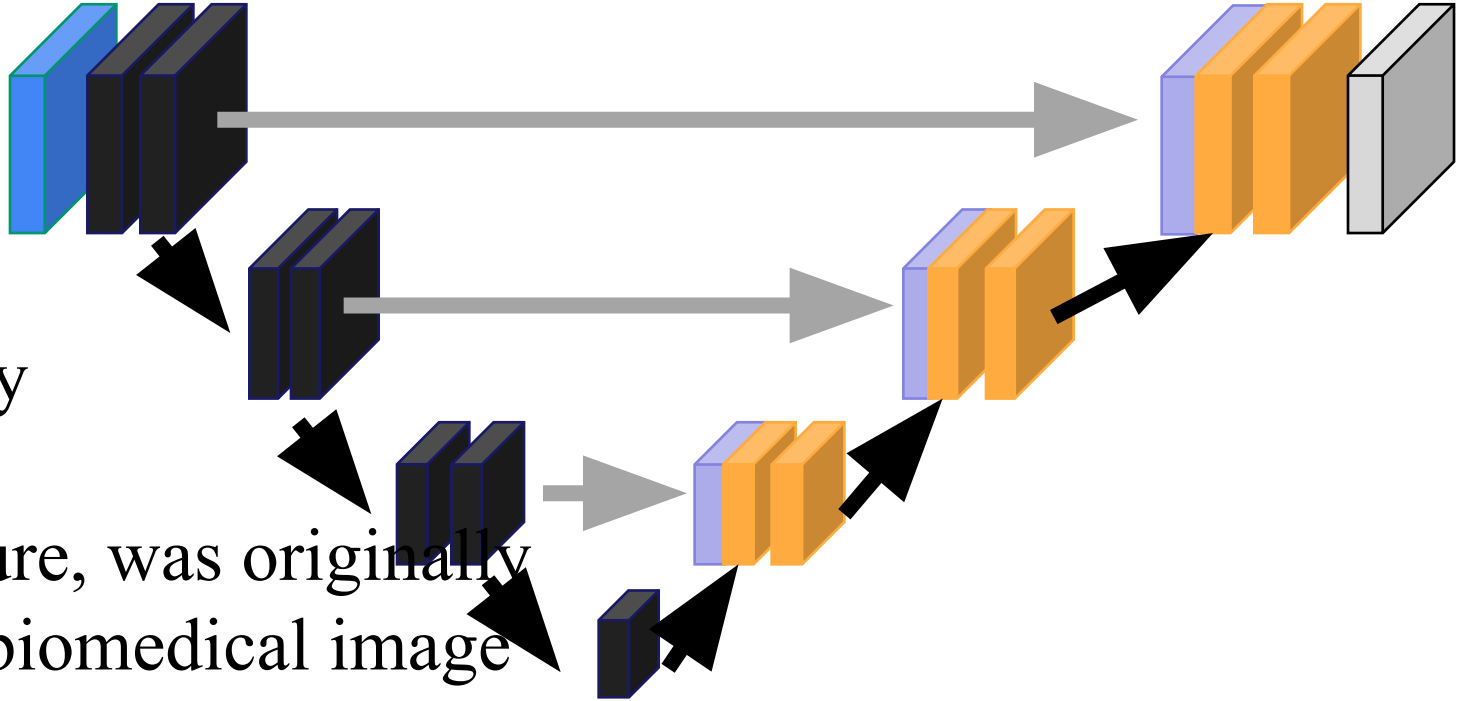
How do you send details forward in the network?
You copy the activations forward.
Subsequent layers at the same resolution figure out how to fuse things.



Copy

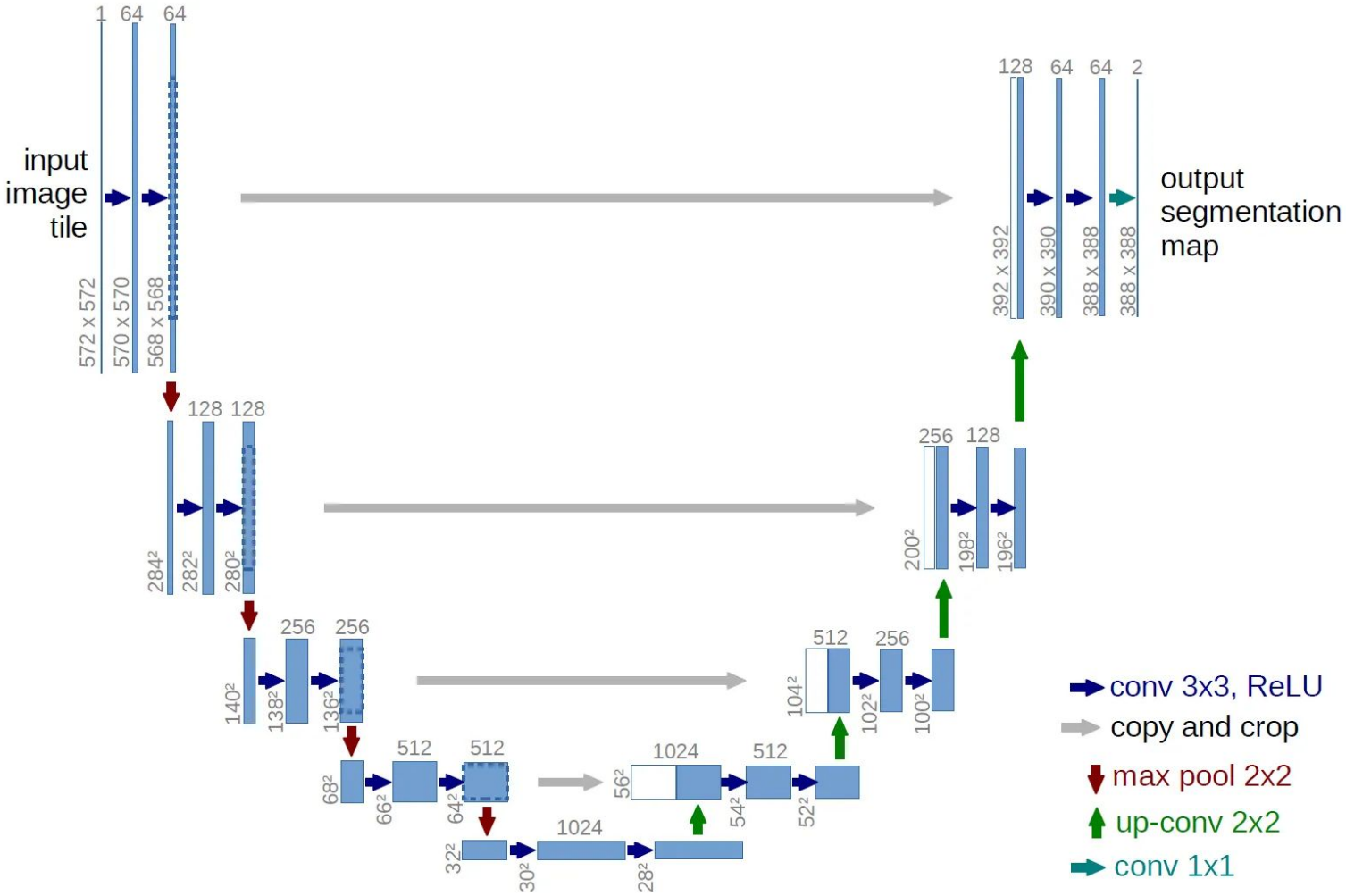
Result from Long et al. *Fully Convolutional Networks For Semantic Segmentation*. CVPR 2014

U-Net

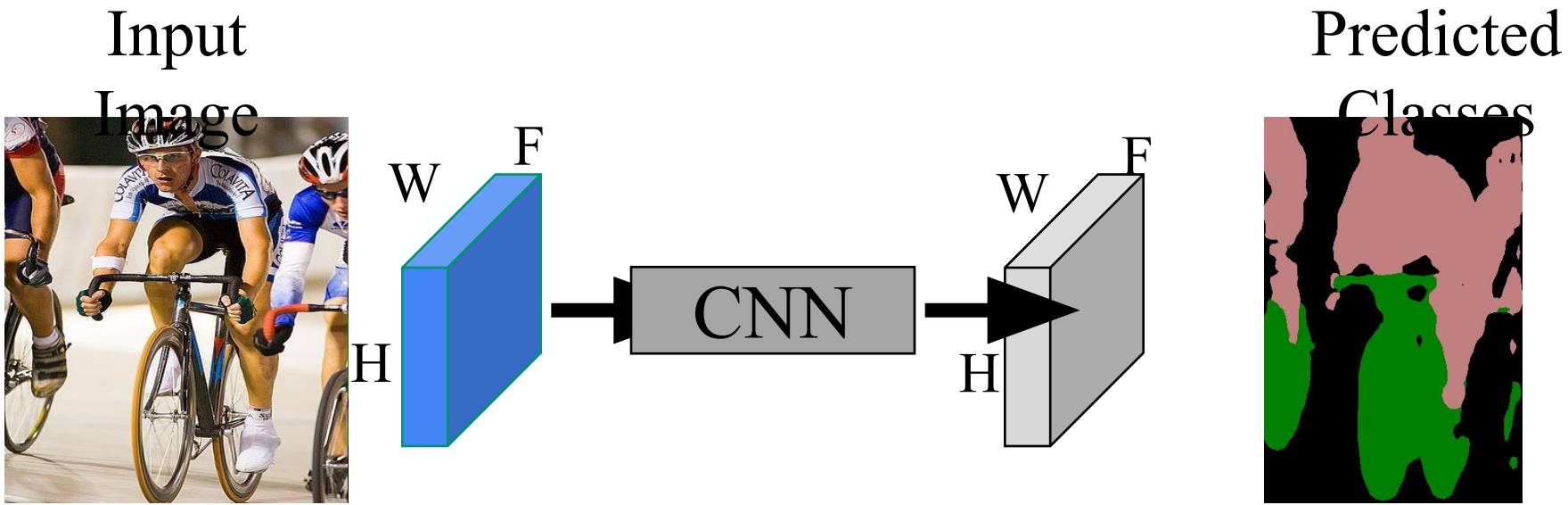


Extremely popular architecture, was originally used for biomedical image segmentation.

Ronneberger et al. “U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015


input image tile
output segmentation map
1 64 64
572 x 572
570 x 570
568 x 568
128 128
284²
282²
280²
256 256
140²
138²
136²
512 512
68²
66²
64²
1024
32²
30²
28²
1024 512
56²
54²
52²
512 256
104²
102²
100²
256 128
200²
198²
196²
128 64 64 2
392 x 392
390 x 390
388 x 388
388 x 388
conv 3x3, ReLU
copy and crop
max pool 2x2
up-conv 2x2
conv 1x1

Evaluating Pixel Labels



How do we convert final HxWxF into labels?

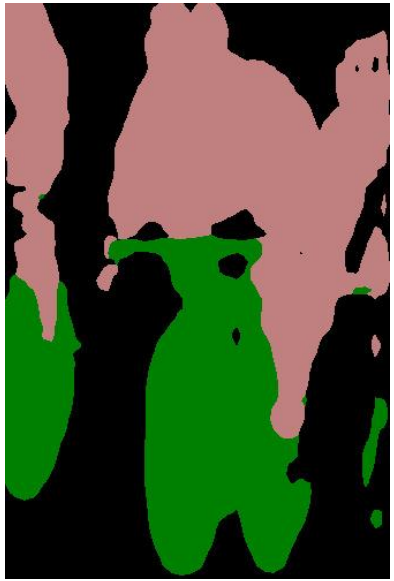
argmax over labels

Evaluating Semantic Segmentation

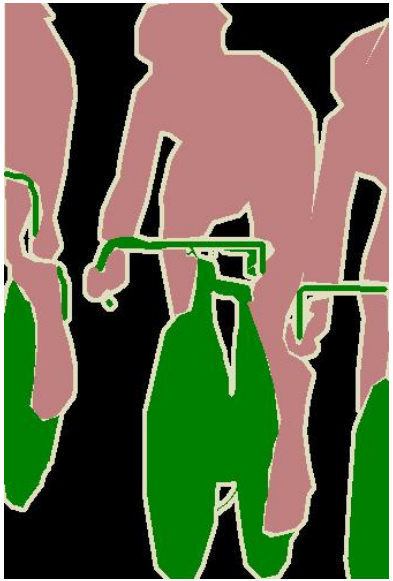
Given predictions, how well did we do?

Input

Prediction ($\hat{y}$)



Ground-Truth (y)

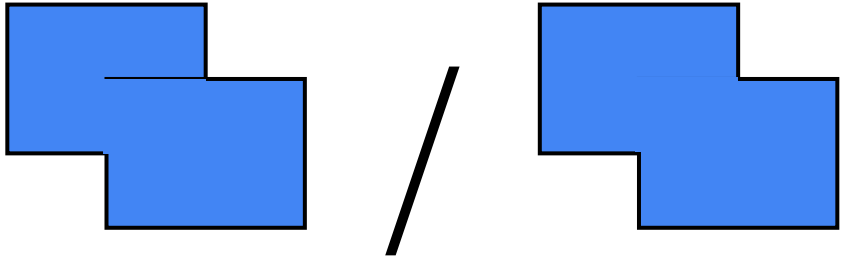


Evaluating Semantic Segmentation

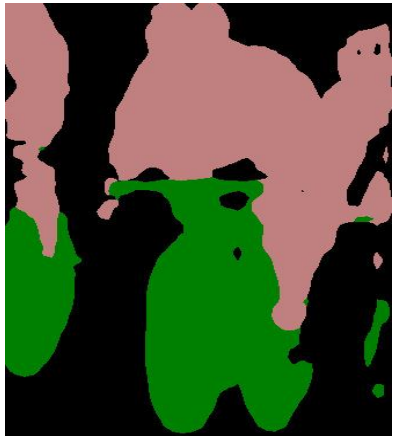
Prediction and ground-truth are images where each pixel is one of F classes.

Accuracy: $\text{mean}(\hat{\boldsymbol{y}} = \boldsymbol{y})$

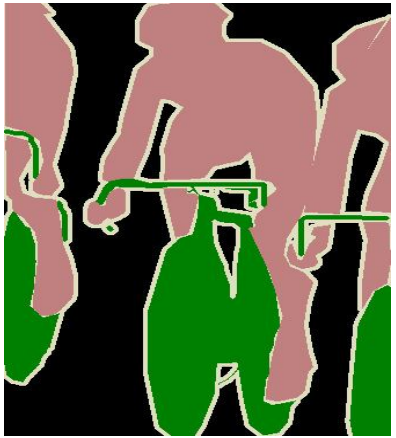
Intersection over union, averaged over classes



Prediction ($\hat{\boldsymbol{y}}$)



Ground-Truth ($\boldsymbol{y}$)



Aside.

Why “Transpose Convolution”?

Can write convolution as matrix-multiply

Input: 4, Filter: 3, Stride: 1, Pad: 1

a	b	c	d

$*$

x	y	z

x	y	z	0	0	0
0	x	y	z	0	0
0	0	x	y	z	0
0	0	0	x	y	z

X

0
a
b
c
d
0

=

ay+bz
ax+by+cz
bx+cy+dz
cx+dy

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Why “Transpose Convolution”?

Transpose convolution is convolution transposed

$$\begin{bmatrix} a & b & c & d \end{bmatrix} *^{T} \begin{bmatrix} x & y & z \end{bmatrix}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \mathbf{X} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay+bx \\ az+by+cx \\ bz+cy+dx \\ cz+dy \\ dz \end{bmatrix}$$

$$\begin{array}{cc} ax & \\ ay+ & bx \\ az+ & by+ \\ & bz+ \end{array} \quad \begin{array}{c} \cdots \\ \cdots \\ \cdots \end{array}$$

Example Credit: L. Fei-Fei, J. Johnson, S. Yeung

Performance Metrics for Segmentation - see materials

mAP for Object Detection

Source: Karel Hora
Brno University of Technology / Czech Technical University in Prague
horak@feec.vutbr.cz

mAP for Object Detection

- mAP stands for mean Average Precision.
- AP is a popular metric in measuring the accuracy of object detectors like Faster R-CNN, SSD, etc.
- Averageprecision computes the average precision valuefor recallvalueover 0 to 1. It sounds complicated but actually pretty simple as we illustrate it with an example.
- But before that, we will do a quick recap on precision, recall, and IoU first.

Precision & recall

- **Precision** measures how accurate is your predictions. i.e. the percentage of your predictions are correct.

- **Recall** measures how good you find all the positives. For example, we can find 80% of the possible positive cases in our top K predictions.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

TP = True positive

TN = True negative

FP = False positive

FN = False negative

- **F1 score** is a measure combining Precision and Recall.

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

		Ground truth	
		FTU (1)	**Background (0)**
Prediction	**FTU (1)**	TP	FP
	Background (0)	FN	TN

Let's interpret these terms in our context! Our goal in this challenge is to segment a mask of (Functional Tissue Units) FTUs for each histopathological image. Each mask is represented either by 0-background or 1-FTU. So,

- **TP (True Positive):** represents the number of FTU pixels that have been properly classified as FTU
- **FP (False Positive):** represents the number background pixels being misclassified as FTUs (due to misalignment)
- **FN (False Negative):** represents the number of FTU pixels being misclassified as background
- **TN(True Negative):** represents the number of background pixels that have been properly classified as background

Precision & Recall:

For pixel classification:

Precision score is the number of true positive results divided by the number of all positive results

$$Precision = \frac{TP}{TP + FP}$$

Recall score, also known as *Sensitivity* or *true positive rate*, is the number of true positive results divided by the number of all samples that should have been identified as positive

$$Recall = \frac{TP}{TP + FN}$$

```
def precision_score_(groundtruth_mask, pred_mask):
    intersect = np.sum(pred_mask*groundtruth_mask)
    total_pixel_pred = np.sum(pred_mask)
    precision = np.mean(intersect/total_pixel_pred)
    return round(precision, 3)

def recall_score_(groundtruth_mask, pred_mask):
    intersect = np.sum(pred_mask*groundtruth_mask)
    total_pixel_truth = np.sum(groundtruth_mask)
    recall = np.mean(intersect/total_pixel_truth)
    return round(recall, 3)
```

https://medium.com/@nghihuynh_37300/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f

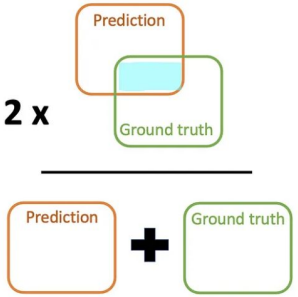
$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$

```
def accuracy(groundtruth_mask, pred_mask):
    intersect = np.sum(pred_mask*groundtruth_mask)
    union = np.sum(pred_mask) + np.sum(groundtruth_mask) - intersect
    xor = np.sum(groundtruth_mask==pred_mask)
    acc = np.mean(xor/(union + xor - intersect))
    return round(acc, 3)
```

Dice coefficient = **F1 score**: a harmonic mean of precision and recall. In other words, it is calculated by 2*intersection divided by the total number of pixel in both images. Note that it's possible to adjust the F-score to give more importance to precision over recall, or vice-versa. Refer to F-beta score for more info.

$$Dice = \frac{2TP}{2TP + FP + FN}$$

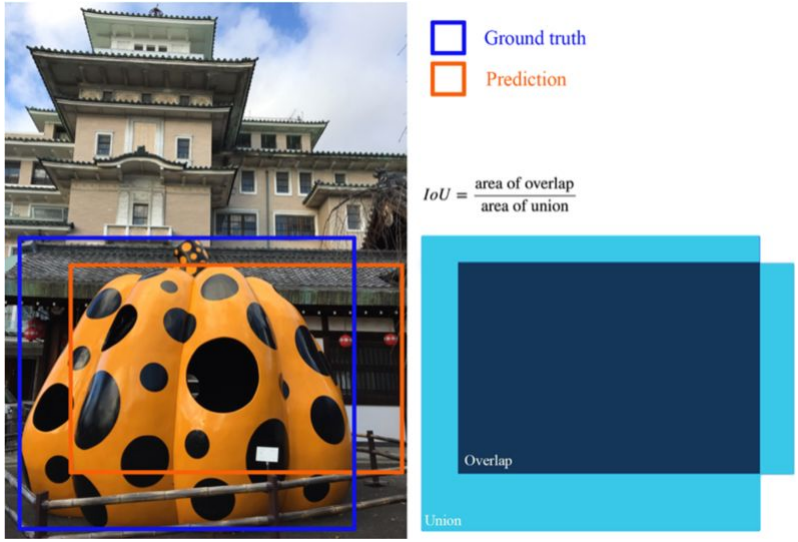
$$\text{Dice} = \frac{2\ X\ Area\ of\ overlap}{Total\ area} =$$




```
def dice_coef(groundtruth_mask, pred_mask):
    intersect = np.sum(pred_mask*groundtruth_mask)
    total_sum = np.sum(pred_mask) + np.sum(groundtruth_mask)
    dice = np.mean(2*intersect/total_sum)
    return round(dice, 3) #round up to 3 decimal places
```

IoU (Intersection over union)

- **IoU** measures the overlap between two boundaries.
- We use that to measure how much our predicted boundary overlaps with the ground truth (the real object boundary).
- In some datasets, we predefine an IoU threshold (say 0.5) in classifying whether the prediction is a true positive or a false positive.

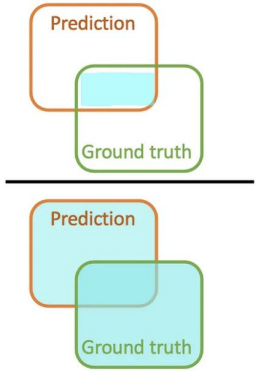


Jaccard Index (IoU):

Jaccard index, also known as **Intersection over Union (IoU)**, is the area of the intersection over union of the predicted segmentation and the ground truth

$$IoU = \frac{TP}{TP + FP + FN}$$

$$\text{IoU} = \frac{Area\ of\ overlap}{Area\ of\ union} =$$




```
def iou(groundtruth_mask, pred_mask):
    intersect = np.sum(pred_mask*groundtruth_mask)
    union = np.sum(pred_mask) + np.sum(groundtruth_mask) - intersect
    iou = np.mean(intersect/union)
    return round(iou, 3)
```

Average Precision

- Let us create an over-simplified example in demonstrating the calculation of the average precision.
- In this example, the whole dataset contains five apples only. We collect all the predictions made for apples in all the images and rank it in descending order according to the predicted confidence level. The second column indicates whether the prediction is correct or not. In this example, the prediction is correct if IoU $\geq$ 0.5.

Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0

Average Precision

- Let us take the row with rank #3 and demonstrate how precision and recall are calculated first.
- **Precision** is the proportion of TP = 2/3 = 0.67.
- **Recall** is the proportion of TP out of the possible positives = 2/5 = 0.4.
- Recall values increase as we go down the prediction ranking. However, precision has a zigzag pattern — it goes down with false positives and goes up again with true positives.

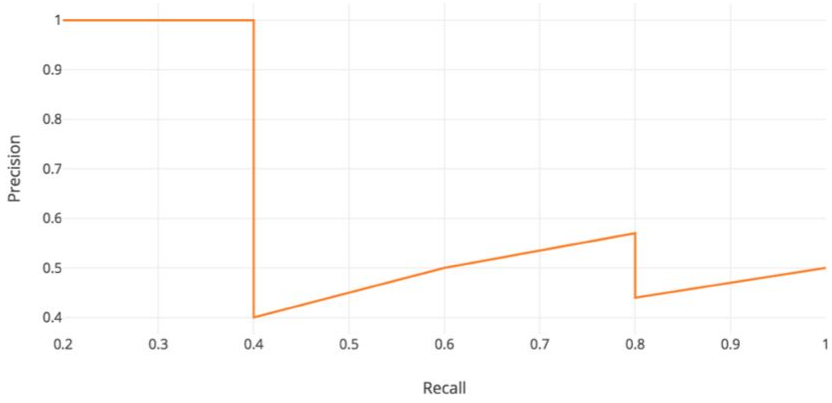
Rank	Correct?	Precision	Recall
1	True	1.0 ↑	0.2 ↑
2	True	1.0 –	0.4 ↑
3	False	0.67 ↓	0.4 –
4	False	0.5 ↓	0.4 –
5	False	0.4 ↓	0.4 –
6	True	0.5 ↑	0.6 ↑
7	True	0.57 ↑	0.8 ↑

Average Precision

- Let us plot the precision against the recall value to see this zig-zag pattern.

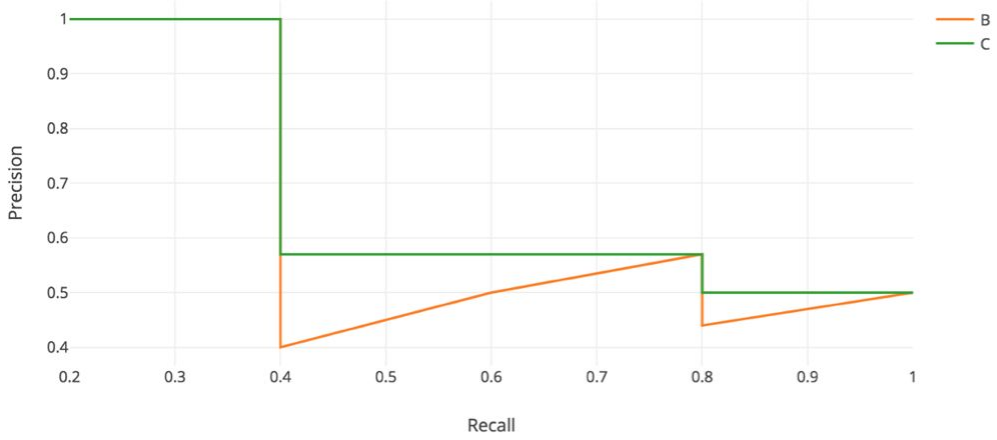
- The general definition for the Average Precision (AP) is finding the area under the precision-recall curve.

$$\mathrm{AP} = \int_0^1 p(r)dr$$



Average Precision

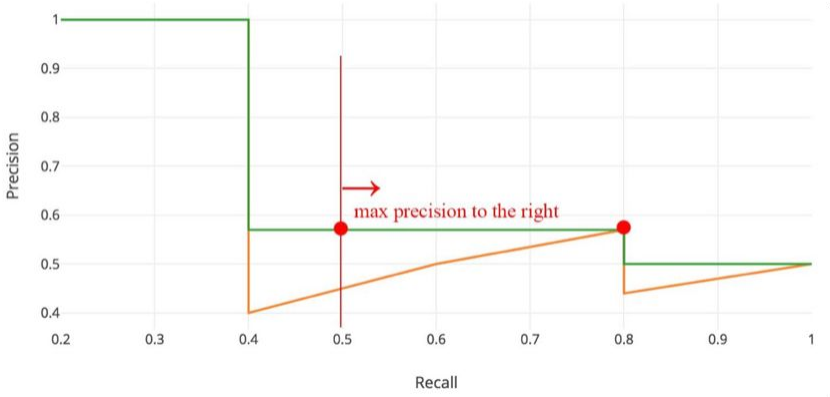
- Precision and recall are always between 0 and 1. Therefore, AP falls within 0 and 1 also.
- Before calculating AP for the object detection, we often smooth out the zigzag pattern first.
- Graphically, at each recall level, we replace each precision value with the maximum precision value to the right of that recall level.



Average Precision

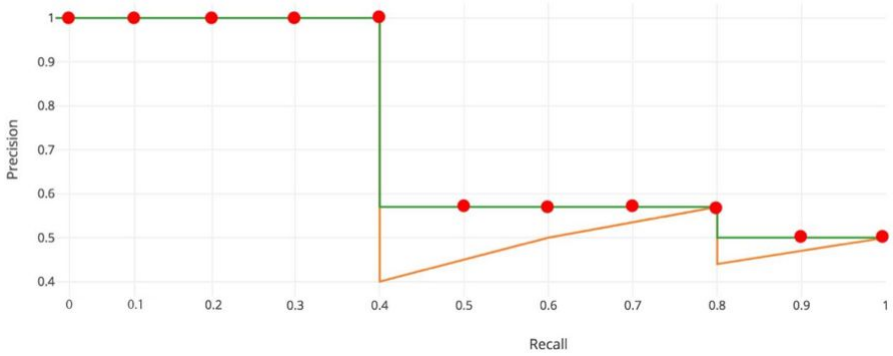
- Sothe orange line is transformed into the green line and the curve will decrease monotonically instead of the zigzag pattern.
- The calculated AP value will be less suspectable to small variations in the ranking.
- Mathematically, we replace the precision value for recall $\hat{r}$ with the maximum precision for any recall $\geq \hat{r}$.

$$p_{interp}(r) = \max_{\tilde{r} \geq r} p(\tilde{r})$$



Interpolated AP

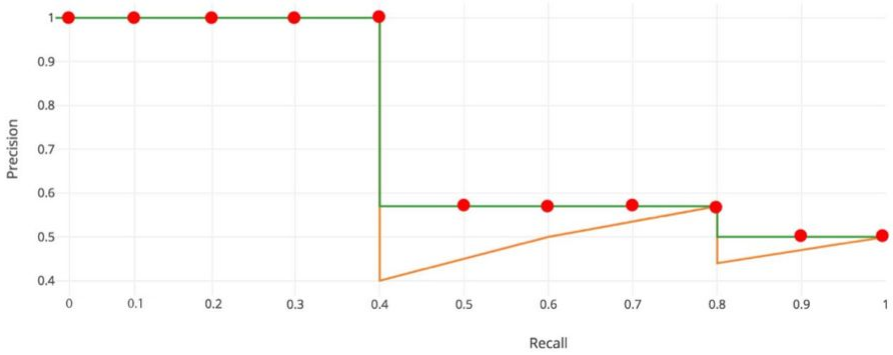
- **PASCAL VOC** is a popular dataset for object detection.
- For the PASCAL VOC challenge, a prediction is positive if IoU $\geq$ 0.5. Also, if multiple detections of the same object are detected, it counts the first one as a positive while the rest as negatives.
- In Pascal VOC2008, an average for the 11-point interpolated AP is calculated.



Interpolated AP

- First, we divide the recall value from 0 to 1.0 into 11 points — 0, 0.1, 0.2, …, 0.9 and 1.0.
- Next, we compute the average of maximum precision value for these 11 recall values.

$$AP = \frac{1}{11} \times \left(AP_r(0) + AP_r(0.1) + \ldots + AP_r(1.0)\right)$$



- In our example, AP = (5 × 1.0 + 4 × 0.57 + 2 × 0.5)/11

Interpolated AP

- When AP_r turns extremely small, we can assume the remaining terms to be zero. i.e. we do not necessarily make predictions until the recall reaches 100%.

- If the possible maximum precision levels drop to a negligible level, we can stop.

- For 20 different classes in PASCAL VOC, we compute an AP for every class and also provide an average for those 20 AP results.

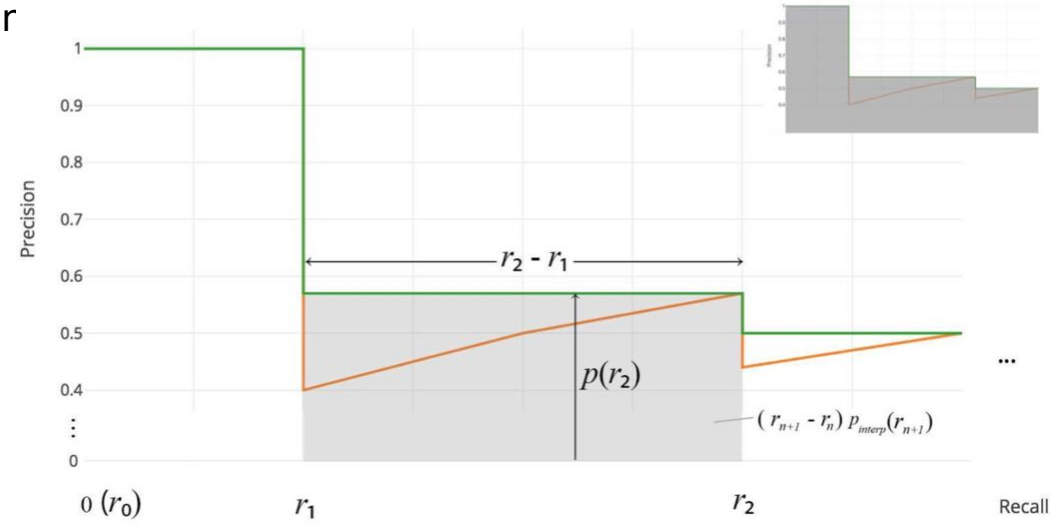
- However, this interpolated method is an approximation which suffers two issues:

1. It is less precise.
2. Second, it lost the capability in measuring the difference for methods with low AP.

- Therefore, a different AP calculation is adopted after 2008 for PASCAL VOC.

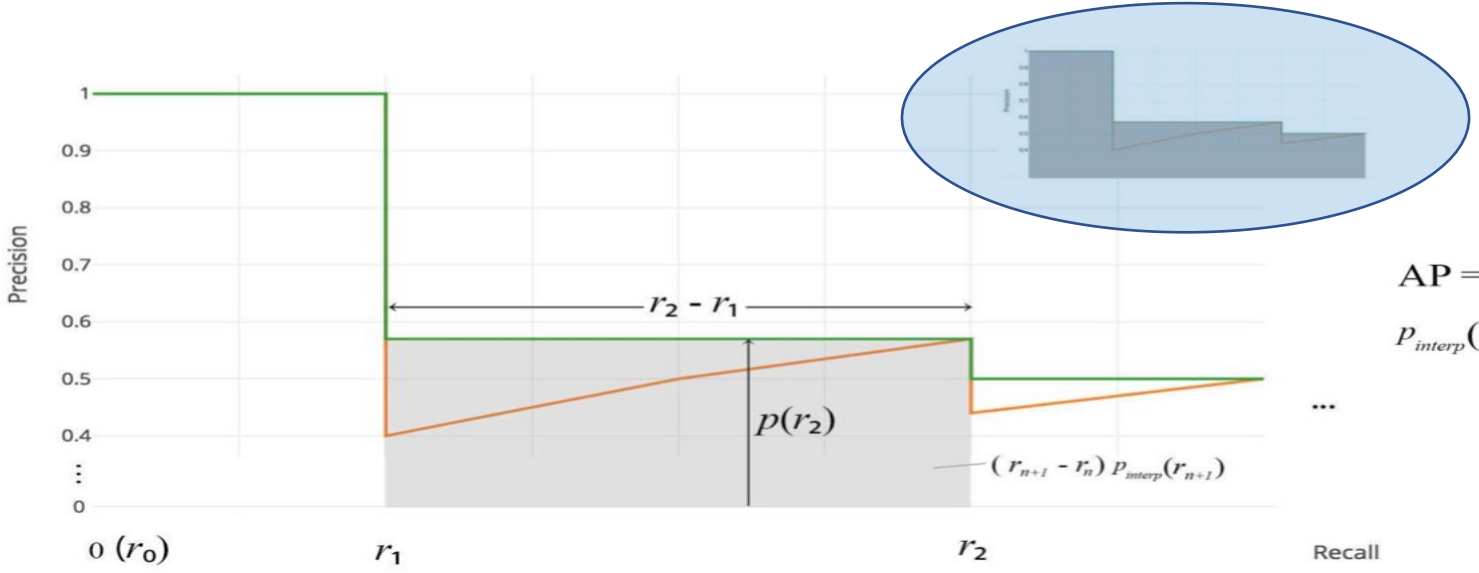
AP (Area under curve AUC)

- For later Pascal VOC competitions, VOC2010–2012 samples the curve at all unique recall values (r_1, r_2, …), whenever the maximum precision value drops.
- With this change, we are r ıfter the zigzags are removed.



AP (Area under curve AUC)

- No approximation or interpolation is needed.
- Instead of sampling 11 points, we sample $p(r_i)$ whenever it drops and computes AP as the sum of the rectangular blocks.

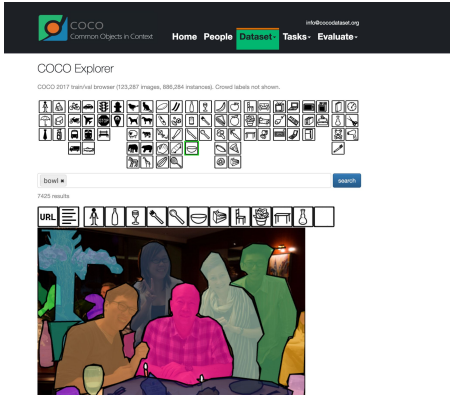


$$AP = \Sigma\, (r_{n+1} - r_n)\, p_{interp}(r_{n+1})$$

$$p_{interp}(r_{n+1}) = \max_{\tilde{r} \geq r_{n+1}} p(\tilde{r})$$

COCO mAP

- Latest research papers tend to give results for the COCO dataset only. In COCO mAP, a 101-point interpolated AP definition is used in the calculation.
- For COCO, AP is the average over multiple IoU (the minimum IoU to consider a positive match).
- **AP@[.5:.95]** corresponds to the average AP for IoU from 0.5 to 0.95 with a step size of 0.05.
- For the COCO competition, AP is the average over 10 IoU levels on 80 categories (AP@[.50:.05:.95]: start from 0.5 to 0.95 with a step size of 0.05).



COCO mAP

- The following are some other metrics collected for the COCO dataset:

```
Average Precision (AP):
   AP                    % AP at IoU=.50:.05:.95 (primary challenge metric)
   AP^{IoU=.50}          % AP at IoU=.50 (PASCAL VOC metric)
   AP^{IoU=.75}          % AP at IoU=.75 (strict metric)
AP Across Scales:
   AP^{small}            % AP for small objects: area < 32^2
   AP^{medium}           % AP for medium objects: 32^2 < area < 96^2
   AP^{large}            % AP for large objects: area > 96^2
Average Recall (AR):
   AR^{max=1}            % AR given 1 detection per image
   AR^{max=10}           % AR given 10 detections per image
   AR^{max=100}          % AR given 100 detections per image
AR Across Scales:
   AR^{small}            % AR for small objects: area < 32^2
   AR^{medium}           % AR for medium objects: 32^2 < area < 96^2
   AR^{large}            % AR for large objects: area > 96^2
```

COCO mAP

- Example: the AP result for the YOLOv3 detector:

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [3]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [6]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [4]	Inception-ResNet-v2 [19]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [18]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [13]	DarkNet-19 [13]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [9, 2]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [2]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [7]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [7]	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

COCO for YOLOv3

- For clarity, AP@.75 means the AP with IoU=0.75.

COCO mAP

- **mAP** (mean average precision) is the average of AP.
- In some context, we compute the AP for each class and average them. But in some context, they mean the same thing. For example, under the COCO context, there is no difference between AP and mAP. Here is the direct quote from COCO:

„AP is averaged over all categories. Traditionally, this is called "mean average precision" (mAP). We make no distinction between AP and mAP (and likewise AR and mAR) and assume the difference is clear from context."

- In ImageNet, the AUC method is used.
- To summarize: even all of them follow the same principle in measurement AP, the exact calculation may vary according to the datasets. Fortunately, development kits are available in calculating this metric.

References

https://www.kaggle.com/code/dschettler8845/visual-guide-to-understanding-segmentation-metrics
https://github.com/bnsreenu/python_for_microscopists/
https://towardsdatascience.com/metrics-to-evaluate-your-semantic-segmentation-model-6bcb99639aa2
https://cocodataset.org/#home
https://www.apeer.com/annotate
https://github.com/mithunkumarsr/LearnComputerVisionWithMithun/blob/main/DL_C4_week3_2_Image_Segmentation_with_U-Net_v2.ipynb
https://www.v7labs.com/blog/mean-average-precision

Thank you